

Graphics Card Market Analysis - Web Scraping Project

Project Overview

In this project, you will create a web scraper to collect and analyze graphics card data from Newegg.com's GPU section. You'll use Python, BeautifulSoup, and other relevant libraries to extract key information about graphics cards currently available in the market.

Learning Objectives

- Implement web scraping techniques using Python and BeautifulSoup
- Handle pagination in web scraping
- Process and structure collected data
- Practice working with real-world data
- Learn to respect website scraping policies and implement delays

Part 1: Web scrapping

Required Data to Extract

For each graphics card listing, you must collect:

1. Product Image URL
2. Product Title (e.g., "MSI Ventus GeForce RTX 3060 12GB")
3. Product Details:
 - Manufacturer
 - Model Number
 - Rating (number of stars)
4. Pricing Information:
 - Current Price
 - Original Price (if available)
 - Shipping Information

 Screenshot

Project Requirements

Technical Requirements

1. Use Python 3.x
2. Required libraries:
 - requests
 - beautifulsoup4
 - pandas
 - time (for implementing delays)

Functional Requirements

1. Implement proper error handling
2. Include appropriate delays between requests (minimum 2 seconds)
3. Save the scraped data to a CSV file with appropriate column headers
4. Document your code with comments
5. Create a summary of the collected data (e.g., average price, price range)

Part 2: Data Analysis

Required Analysis

1. Price Analysis
 - Calculate average price by brand
 - Identify price ranges and market segments
 - Analyze price-to-memory ratio
 - Compare prices across different GPU series (RTX 3000 vs 4000)
 - Create price distribution visualizations
2. Brand Analysis
 - Market share by brand
 - Average ratings by brand
 - Price comparison across brands
 - Create brand comparison visualizations
3. Memory Configuration Analysis
 - Distribution of memory sizes
 - Price correlation with memory size
 - Popular memory configurations by brand
4. Market Segmentation
 - Entry-level vs Mid-range vs High-end segments
 - Price brackets and their features

Deliverables

1. Python script containing the web scraper
2. CSV file with the collected data
3. README file explaining:
 - Project Description
 - Features
4. Short report analyzing the collected data

Important Notes

- Always check and respect the website's robots.txt file
- Implement appropriate delays between requests to avoid overwhelming the server
- Handle your requests responsibly and ethically
- Be prepared to handle cases where data might be missing or in an unexpected format

```
In [2]: # Import chrome webdriver to access chrome options
# Import chrome webdriver to access chrome options
from webdriver_manager.chrome import ChromeDriverManager
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.common.by import By
import pandas as pd
import time
import selenium
from selenium import webdriver
```

```
In [15]: url= 'https://www.newegg.com/GPUs-Video-Graphics-Cards/SubCategory/ID-48'
service = Service(ChromeDriverManager().install())
driver = webdriver.Chrome(service=service)
driver.get(url)
time.sleep(3)
cards= driver.find_elements(By.CSS_SELECTOR , 'div.item-cell')
data =[]
#Loop through each card and extract fields
for card in cards:
    ##Field 1
    try:
        image_url = card.find_element(By.CSS_SELECTOR , 'a.item-img img').get_attri
    except:
        image_url = None
    ##Field 2 Rating
    try:
        rating = card.find_element(By.CSS_SELECTOR , 'a.item-rating').get_attribute
    except:
        rating= None
    try :
        review = card.find_element(By.CSS_SELECTOR , 'span.item-rating-num').text.s
    except:
        review = None
    #Field 3 : Manufacturer (title attribute, fallback alt)
    try:
        brand_img=card.find_element(By.CSS_SELECTOR , 'a.item-brand img')
```

```

        brand = brand_img.get_attribute('title') or brand_img.get_attribute("alt")
    except:
        brand = None
#Field 4 Product Title
    try:
        title= card.find_element(By.CSS_SELECTOR , 'a.item-title').text
    except:
        title = None
## Field 5: Model number (split text after "Model #:")
    try:
        features_text = card.find_element(By.CSS_SELECTOR , 'ul.item-features').text
        model =features_text.split("Model #:")[-1].strip() if "Model #:" in features_text
    except:
        model = None
## Field 6 Current Price (dollars + cents combined)
    try:
        price_el = card.find_element(By.CSS_SELECTOR , 'li.price-current')
        dollars =price_el.find_element(By.TAG_NAME , "strong")
        cents=price_el.find_element(By.TAG_NAME , "sup")
        price =f"{dollars.text}{cents.text}"
    except:
        price = None
## Field 7: shipping info
    try:
        shipping = card.find_element(By.CSS_SELECTOR, 'li.price-ship').text
    except:
        shipping = None

## Save this card's data (runs for every card, not just failures)
    data.append({
        "Title": title, "Brand": brand, "Rating": rating,
        "Review_Count": review, "Model": model, "Price": price,
        "Shipping": shipping, "Image_URL": image_url
    })

df = pd.DataFrame(data)
print(df)
df.to_csv("newegg_products.csv", index=False)
driver.quit()

```

	Title	Brand \
0	SAPPHIRE PULSE Radeon RX 9060 XT 16GB GDDR6 PC...	Sapphire Tech
1	ASRock Challenger Radeon RX 9060 XT 16GB GDDR6...	ASRock
2	GIGABYTE Gaming GeForce RTX 5070 Ti 16GB GDDR7...	GIGABYTE
3	ASRock Challenger Radeon RX 9070 XT 16GB GDDR6...	ASRock
4	ASRock Challenger Radeon RX 9070 16GB GDDR6 PC...	ASRock
5	GIGABYTE Gaming Radeon RX 9070 GRE 12GB GDDR6 ...	GIGABYTE
6	ASRock Challenger Arc B580 12GB GDDR6 PCI Expr...	ASRock
7	GIGABYTE Gaming GeForce RTX 5080 16GB GDDR7 PC...	GIGABYTE
8	GIGABYTE Gaming GeForce RTX 5070 12GB GDDR7 PC...	GIGABYTE
9	MSI FRIEREN EDITION GeForce RTX 5070 Ti 16GB G...	MSI
10	XFX Quicksilver AMD Radeon RX 9070 XT Gaming E...	XFX
11	SAPPHIRE PULSE Radeon RX 9070 XT 16GB GDDR6 PC...	Sapphire Tech
12	SAPPHIRE NITRO+ Radeon RX 9070 XT 16GB GDDR6 P...	Sapphire Tech
13	ASRock Challenger Radeon RX 7600 8GB GDDR6 PCI...	ASRock
14	ASRock Radeon RX 7900 XTX 24GB GDDR6 PCI Expre...	ASRock
15	GIGABYTE WindForce GeForce RTX 5070 Ti 16GB GD...	GIGABYTE
16	GIGABYTE Gaming Radeon RX 9070 XT 16GB GDDR6 P...	GIGABYTE
17	ONIX LUMI Arc B580 12GB GDDR6 PCI Express 4.0 ...	None
18	GIGABYTE WindForce GeForce RTX 5070 12GB GDDR7...	GIGABYTE
19	MSI Ventus GeForce RTX 5060 Ti 8GB GDDR7 PCI E...	MSI
20	MSI SHADOW GeForce RTX 5080 16GB GDDR7 PCI Exp...	MSI
21	GIGABYTE Eagle GeForce RTX 5070 12GB GDDR7 PCI...	GIGABYTE
22	MSI Gaming GeForce RTX 5090 32GB GDDR7 PCI Exp...	MSI
23	MSI Ventus GeForce RTX 5060 Ti 16GB GDDR7 PCI ...	MSI
24	GIGABYTE WINDFORCE GeForce RTX 5080 16GB GDDR7...	GIGABYTE
25	ASRock Steel Legend Radeon RX 9070 XT 16GB GDD...	ASRock
26	GIGABYTE Eagle GeForce RTX 5070 Ti 16GB GDDR7 ...	GIGABYTE
27	GIGABYTE Gaming Radeon RX 9060 XT 16GB GDDR6 P...	GIGABYTE
28	MSI LP GeForce RTX 3050 6GB GDDR6 PCI Express ...	MSI
29	XFX Swift AMD Radeon RX 9060 XT OC Gaming Edit...	XFX
30	ASUS ROG Astral GeForce RTX 5090 32GB GDDR7 OC...	ASUS
31	GIGABYTE Gaming GeForce RTX 5090 32GB GDDR7 PC...	GIGABYTE
32	GIGABYTE Gaming Radeon RX 9060 XT 16GB GDDR6 P...	GIGABYTE
33	MSI SHADOW GeForce RTX 5070 12GB GDDR7 PCI Exp...	MSI
34	ASUS PRIME GeForce RTX 5070 Ti 16GB 256-Bit GD...	ASUS
35	MSI Gaming GeForce RTX 5070 Ti 16GB GDDR7 PCI ...	MSI

	Rating	Review_Count	Model	Price \
0	Rating + 4.7	125	11350-03-20G	499.99
1	Rating + 4.7	111	RX9060XT CL 16GO	469.99
2	Rating + 4.8	241	GV-N507TGAMING OC-16GD	1,249.99
3	Rating + 4.6	139	RX9070XT CL 16G	821.84
4	Rating + 4.5	74	RX9070 CL 16G	649.99
5	Rating + 4.5	237	GV-R907GREGAMING OC-12GD	549.99
6	Rating + 4.2	54	B580 CL 12GO	309.99
7	Rating + 4.6	301	GV-N5080GAMING OC-16GD	1,699.99
8	Rating + 4.3	142	GV-N5070GAMING OC-12GD	839.99
9	Rating + 4.6	394	RTX 5070 Ti 16G FRIEREN EDITION OC	1,149.99
10	Rating + 5	2	RX-97TQICKBA	749.99
11	Rating + 4.5	158	11348-03-20G	849.99
12	Rating + 4.5	158	11348-01-20G	849.99
13	Rating + 4.6	75	RX7600 CL 8GO	319.99
14	Rating + 4.3	355	RADEON RX7900XTX 24G	1,029.99
15	Rating + 4.8	241	GV-N507TWF30C-16GD	1,169.99
16	Rating + 4.5	237	GV-R9070XTGAMING OC-16GD	810.00

17	Rating + 4.5	153	8346-00278	309.99
18	Rating + 4.3	142	GV-N5070WF30C-12GD	799.99
19	Rating + 4.6	167	RTX 5060 Ti 8G VENTUS 2X OC PLUS	509.99
20	Rating + 4.5	248	RTX 5080 16G SHADOW 3X OC	1,579.99
21	Rating + 4.3	142	GV-N5070EAGLE OC-12GD	809.99
22	Rating + 4.7	136	RTX 5090 32G GAMING TRIO OC	4,899.99
23	Rating + 4.6	167	RTX 5060 Ti 16G VENTUS 3X OC	799.99
24	Rating + 4.6	301	GV-N5080WF30C-16GD	1,599.99
25	Rating + 4.5	182	RX9070XT SL 16G	769.99
26	Rating + 4.8	241	GV-N507TEAGLEOC ICE-16GD	1,189.99
27	Rating + 4.8	158	GV-R906XGAMINGOCICE-16GD	549.99
28	Rating + 5	19	RTX 3050 LP 6G OC	219.99
29	Rating + 4.7	27	RX-96TSW16BQ	509.99
30	Rating + 4.6	97	ROG-ASTRAL-RTX5090-032G-GAMING	5,581.84
31	Rating + 4.3	138	GV-N5090GAMING OC-32GD	4,899.99
32	Rating + 4.8	158	GV-R9060XTGAMING OC-16GD	529.99
33	Rating + 4.7	210	RTX 5070 12G SHADOW 3X OC	809.99
34	Rating + 4.6	251	PRIME-RTX5070TI-16G	1,119.99
35	Rating + 4.6	394	RTX 5070 TI 16G GAMING TRIO OC	1,049.99

	Shipping	Image_URL
0	Free Shipping	https://c1.neweggimages.com/productimage/nb300...
1	Free Shipping	https://c1.neweggimages.com/productimage/nb300...
2	Free Shipping	https://c1.neweggimages.com/productimage/nb300...
3	\$12.99 Shipping	https://c1.neweggimages.com/productimage/nb300...
4	Free Shipping	https://c1.neweggimages.com/productimage/nb300...
5	\$14.65 Shipping	https://c1.neweggimages.com/productimage/nb300...
6	Free Shipping	https://c1.neweggimages.com/productimage/nb300...
7	Free Shipping	https://c1.neweggimages.com/productimage/nb300...
8	Free Shipping	https://c1.neweggimages.com/productimage/nb300...
9	Free Shipping	https://c1.neweggimages.com/productimage/nb300...
10	Free Shipping	https://c1.neweggimages.com/productimage/nb300...
11	Free Shipping	https://c1.neweggimages.com/productimage/nb300...
12	Free Shipping	https://c1.neweggimages.com/productimage/nb300...
13	Free Shipping	https://c1.neweggimages.com/productimage/nb300...
14	Free Shipping	https://c1.neweggimages.com/productimage/nb300...
15	Free Shipping	https://c1.neweggimages.com/productimage/nb300...
16	Free Shipping	https://c1.neweggimages.com/productimage/nb300...
17	Free Shipping	https://c1.neweggimages.com/productimage/nb300...
18	Free Shipping	https://c1.neweggimages.com/productimage/nb300...
19	Free Shipping	https://c1.neweggimages.com/productimage/nb300...
20	Free Shipping	https://c1.neweggimages.com/productimage/nb300...
21	Free Shipping	https://c1.neweggimages.com/productimage/nb300...
22	None	https://c1.neweggimages.com/productimage/nb300...
23	Free Shipping	https://c1.neweggimages.com/productimage/nb300...
24	Free Shipping	https://c1.neweggimages.com/productimage/nb300...
25	Free Shipping	https://c1.neweggimages.com/productimage/nb300...
26	Free Shipping	https://c1.neweggimages.com/productimage/nb300...
27	Free Shipping	https://c1.neweggimages.com/productimage/nb300...
28	\$9.99 Shipping	https://c1.neweggimages.com/productimage/nb300...
29	Free Shipping	https://c1.neweggimages.com/productimage/nb300...
30	\$18.99 Shipping	https://c1.neweggimages.com/productimage/nb300...
31	Free Shipping	https://c1.neweggimages.com/productimage/nb300...
32	Free Shipping	https://c1.neweggimages.com/productimage/nb300...
33	Free Shipping	https://c1.neweggimages.com/productimage/nb300...

34 Free Shipping <https://c1.neweggimages.com/productimage/nb300...>
 35 Free Shipping <https://c1.neweggimages.com/productimage/nb300...>

```
In [16]: import pandas as pd
import matplotlib.pyplot as plt
import re

df = pd.read_csv("newegg_products.csv")

# Cleaning
df['Price'] = df['Price'].astype(str).str.replace('$', '', regex=False).str.replace
df['Price'] = pd.to_numeric(df['Price'], errors='coerce')

df['Rating_Num'] = df['Rating'].astype(str).str.extract(r'([\d.]+)').astype(float)

df['Review_Count'] = df['Review_Count'].astype(str).str.replace(',', '', regex=False)
df['Review_Count'] = pd.to_numeric(df['Review_Count'], errors='coerce')

df['Memory_GB'] = df['Title'].astype(str).str.extract(r'(\d+)\s*GB', flags=re.IGNORECASE)

df['GPU_Series'] = df['Title'].astype(str).str.extract(r'(RTX\s?\d{4}|GTX\s?\d{4})|R
df['GPU_Series'] = df['GPU_Series'].str.upper().str.replace(' ', '', regex=False)

df_clean = df.dropna(subset=['Price']).copy()

print(f"Total products scraped: {len(df)}")
print(f"Products with valid price: {len(df_clean)}")
```

Total products scraped: 36

Products with valid price: 36

```
In [17]: #Brand Analysis
# Market share by brand (% of listings)
market_share = df_clean['Brand'].value_counts(normalize=True) * 100
print("Market share by brand (%):")
print(market_share.round(1))

# Average ratings by brand
avg_rating_by_brand = df_clean.groupby('Brand')['Rating_Num'].mean().sort_values(ascending=True)
print("\nAverage rating by brand:")
print(avg_rating_by_brand)

# Average price by brand (for comparison)
avg_price_by_brand = df_clean.groupby('Brand')['Price'].mean().sort_values(ascending=True)
print("\nAverage price by brand:")
print(avg_price_by_brand)

# Brand comparison visualization
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
avg_price_by_brand.plot(kind='bar', ax=axes[0], title='Avg Price by Brand')
axes[0].set_ylabel("Price ($)")
avg_rating_by_brand.plot(kind='bar', ax=axes[1], title='Avg Rating by Brand', color='red')
axes[1].set_ylabel("Rating")
plt.tight_layout()
plt.savefig("brand_comparison.png")
plt.show()
```

Market share by brand (%):

Brand

GIGABYTE	37.1
MSI	22.9
ASRock	20.0
Sapphire Tech	8.6
XFX	5.7
ASUS	5.7

Name: proportion, dtype: float64

Average rating by brand:

Brand

XFX	4.850000
MSI	4.662500
ASUS	4.600000
GIGABYTE	4.569231
Sapphire Tech	4.566667
ASRock	4.485714

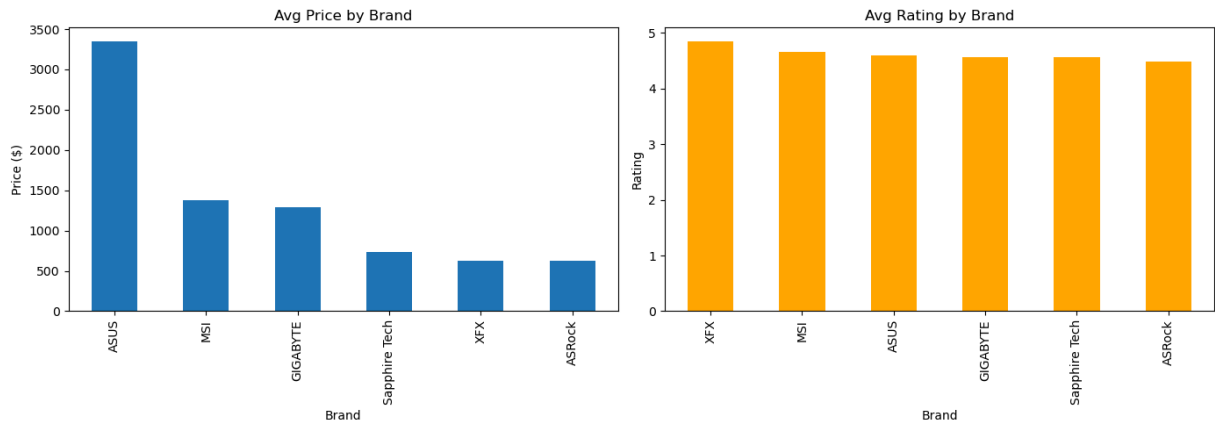
Name: Rating_Num, dtype: float64

Average price by brand:

Brand

ASUS	3350.915000
MSI	1377.490000
GIGABYTE	1284.606154
Sapphire Tech	733.323333
XFX	629.990000
ASRock	624.540000

Name: Price, dtype: float64



```
In [18]: #Memory Configuration Analysis
# Distribution of memory sizes
print("Memory size distribution:")
print(df_clean['Memory_GB'].value_counts().sort_index())

plt.figure(figsize=(8, 5))
df_clean['Memory_GB'].value_counts().sort_index().plot(kind='bar')
plt.title("Distribution of Memory Sizes")
plt.xlabel("Memory (GB)")
plt.ylabel("Count")
plt.tight_layout()
plt.savefig("memory_distribution.png")
plt.show()
```



```
# Price correlation with memory size
correlation = df_clean[['Price', 'Memory_GB']].corr().iloc[0, 1]
print(f"\nCorrelation between Price and Memory size: {correlation:.3f}")

plt.figure(figsize=(8, 5))
plt.scatter(df_clean['Memory_GB'], df_clean['Price'], alpha=0.6)
plt.title("Price vs Memory Size")
plt.xlabel("Memory (GB)")
plt.ylabel("Price ($)")
plt.tight_layout()
plt.savefig("price_vs_memory.png")
plt.show()

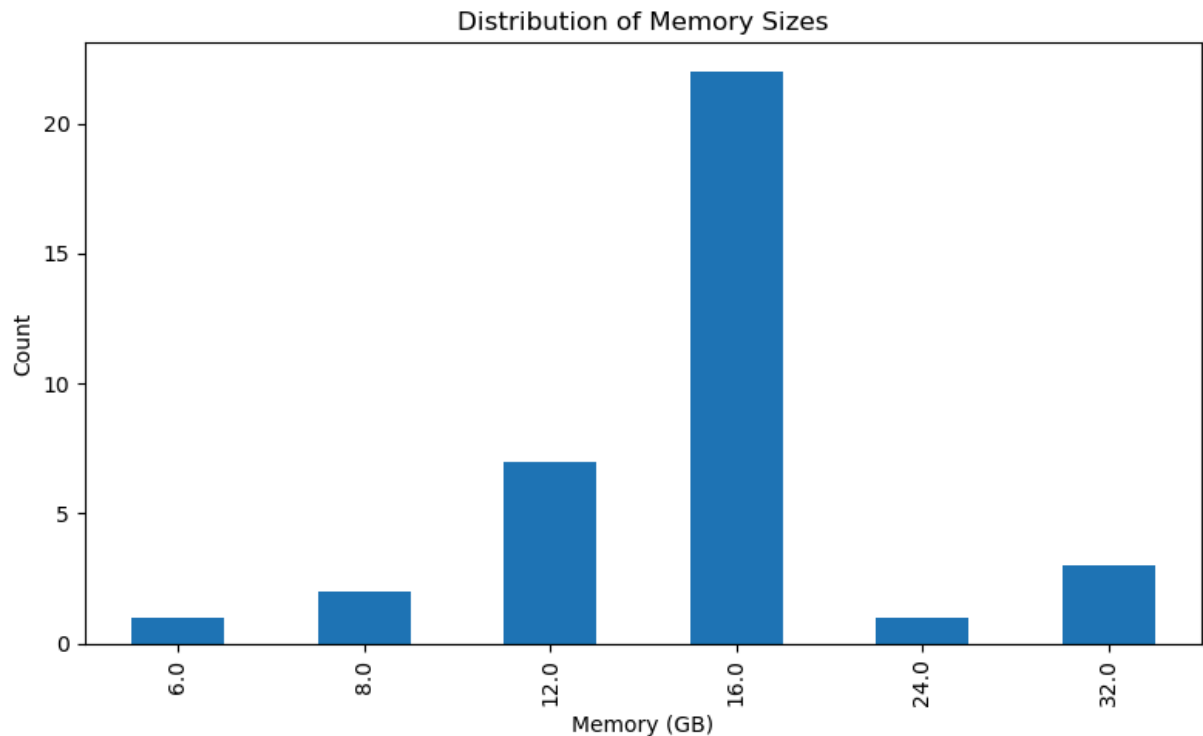
# Popular memory configurations by brand
print("\nMost common memory size by brand:")
print(df_clean.groupby('Brand')['Memory_GB'].agg(lambda x: x.mode().iloc[0] if not
```

Memory size distribution:

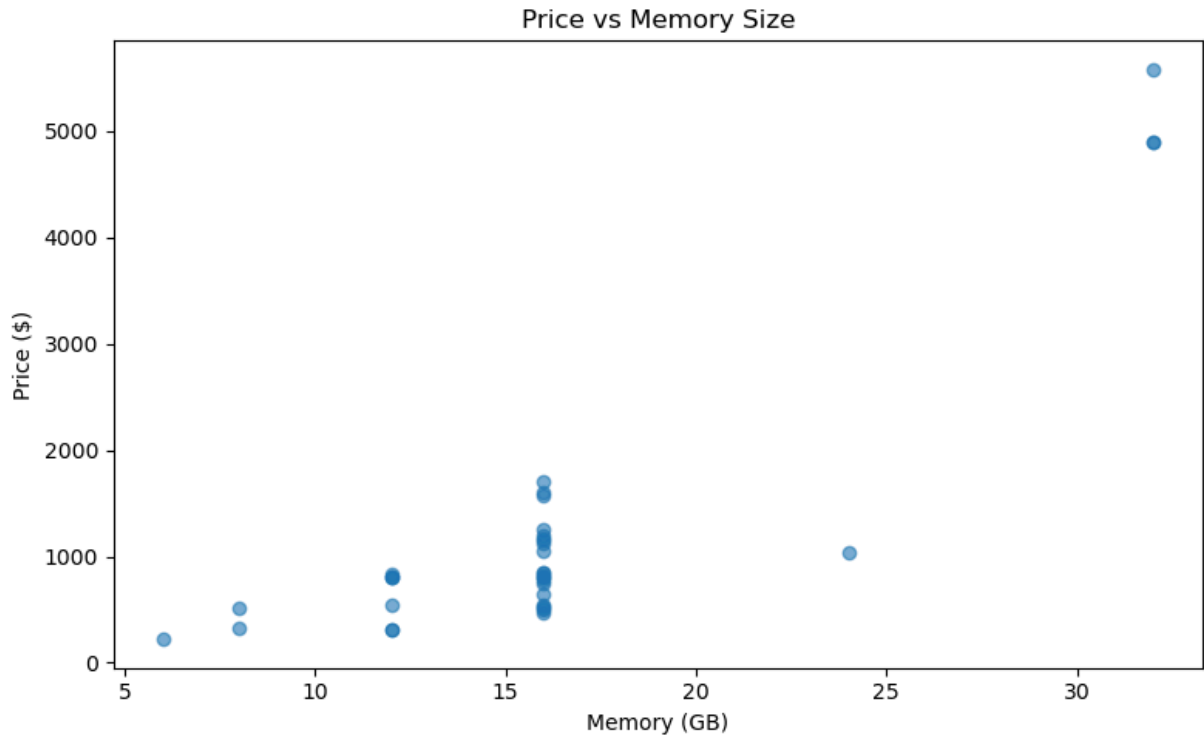
Memory_GB

6.0	1
8.0	2
12.0	7
16.0	22
24.0	1
32.0	3

Name: count, dtype: int64



Correlation between Price and Memory size: 0.882



Most common memory size by brand:

Brand

ASRock 16.0

ASUS 16.0

GIGABYTE 16.0

MSI 16.0

Sapphire Tech 16.0

XFX 16.0

Name: Memory_GB, dtype: float64

```
In [19]: # Define segments by price bracket
def segment(price):
    if price < 300:
        return 'Entry-level'
    elif price < 700:
        return 'Mid-range'
    else:
        return 'High-end'

df_clean['Segment'] = df_clean['Price'].apply(segment)
print("Count per segment:")
print(df_clean['Segment'].value_counts())

print("\nAverage rating and memory per segment:")
print(df_clean.groupby('Segment')[['Rating_Num', 'Memory_GB']].mean())

plt.figure(figsize=(8, 5))
df_clean['Segment'].value_counts().plot(kind='bar', color='green')
plt.title("Products per Price Segment")
plt.ylabel("Count")
plt.tight_layout()
plt.savefig("market_segments.png")
plt.show()
```

```
# Save the enriched dataset with all derived columns
df_clean.to_csv("newegg_products_analyzed.csv", index=False)
print("\nSaved enriched dataset to newegg_products_analyzed.csv")
```

Count per segment:

Segment

High-end 24

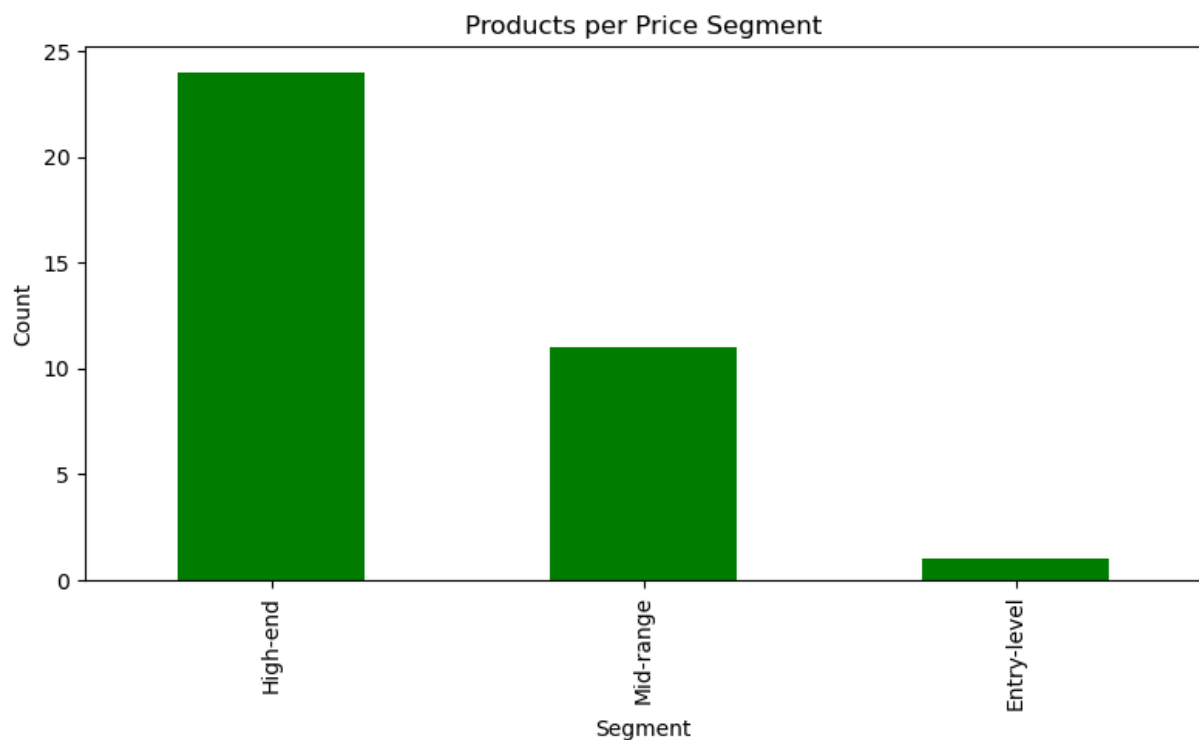
Mid-range 11

Entry-level 1

Name: count, dtype: int64

Average rating and memory per segment:

	Rating_Num	Memory_GB
Segment		
Entry-level	5.000000	6.000000
High-end	4.566667	17.666667
Mid-range	4.600000	13.454545



Saved enriched dataset to newegg_products_analyzed.csv